ECEN/CSEN 123 Lab 2 Report

Tuesday 2:15 - 5:00

Austin Petersen, Dylan Thornburg, Madeline Follosco

I. Objectives

- Detect and process interrupts
- Generate interrupts using a counter/timer module

- Debounce a switch in hardware
- Install a wheel sensors and measure distance using interrupts

III. Lab Procedure

Experiment 1:

1. Insert a pushbutton switch onto your protoboard. Test it first with a multimeter to see how it works and identify which pins do what. You want the switch to close when you push it. Insert it carefully, the leads bend easily. If the leads are not straight – straighten them first with needle nose pliers.

2. Connect the switch between Arduino pin 2 and ground. Arduino pin 2 is connected to interrupt number 0. Program pin 2 to be an input with an internal pull-up resistor.

3. Write a program that triggers an interrupt each time the switch is pressed. Detect the falling edge of the input signal on pin 2. The interrupt service routine should include a counter that increments by 1 each time the interrupt is triggered.

4. Each time the button is pressed, print out the button-press count to the serial console. The serial print code should not be part of the interrupt service routine. It should be in your loop() procedure. You should only print out the number once for each button press. Figure out how to do that. Remember that when you use the serial port, you may need to reset the Arduino after you download the code and open the serial monitor window.

5. Test your program and document for the report. Your program may not work right. You will probably get multiple counts for each button press. Be prepared to explain why.

We got pretty good results, but occasionally the interrupt message or count would multiple times if pressed too fast:

OUTPUT:

```
Count is 42

We have been interrupted.

Count is 43

We have been interrupted.

We have been interrupted.

Count is 45
```

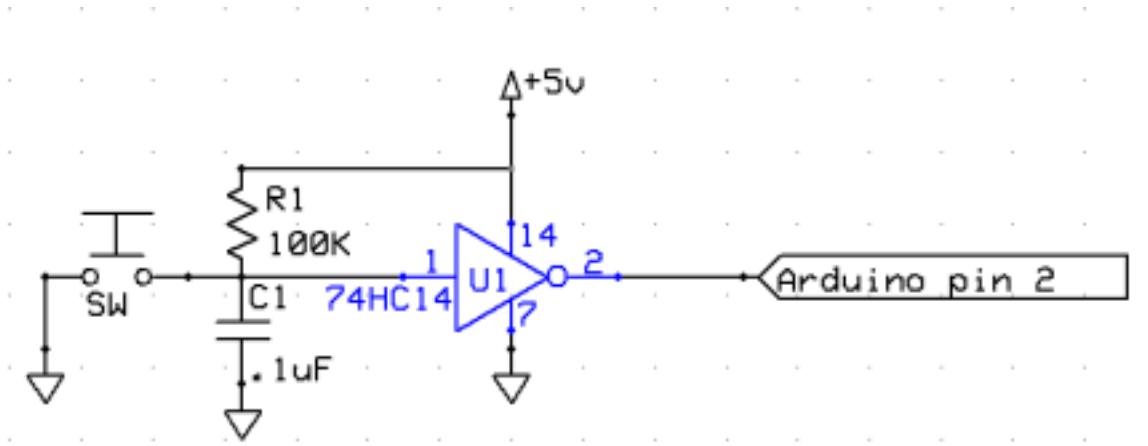
This is because the threshold voltage isn't consistently high or low enough. Adding the 74HC14 will help clean the rising/falling edges.

CODE

```
//demo1
#define PIN_NUM 2
#define ANALOG_PIN A0

void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  pinMode(PIN_NUM, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_NUM), isr0, FALLING);

  pinMode(ANALOG_PIN, INPUT); // Configure pin A0 as an input for
analog reading
}
volatile int flag = 0;
volatile int count = 0;

void loop() {
  if (flag) {
    Serial.println("Count is " + String(count));
    flag = 0;
  }
}

void isr0() {
  flag = 1;
  count++;
  Serial.println("We have been interrupted.");
}
```

Experiment 2:

1. You need to fix the problem with your switch. This requires removing the signal noise. Use the following circuit. Remember to look at the 74HC14 datasheet to understand the pin numbers. Also remember that the 74HC14 needs power.



2. Note that the 74HC14 is an inverter. Change your interrupt code so that the program now detects the rising edge of the signal at pin 2.

3. Retest your program and demo for the TA. (Demo #1). It should now work reliably. If not, use an oscilloscope and a multimeter and figure out why. Do not disconnect this circuit; you need it for experiment 3.

We no longer have the problem where some information is lost if we click too fast. The inverter ensures a strong high or low signal and removes noise.

CODE

```
//demo1
#define PIN_NUM 2
#define ANALOG_PIN A0

void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  pinMode(PIN_NUM, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_NUM), isr0, RISING);

  pinMode(ANALOG_PIN, INPUT); // Configure pin A0 as an input for analog
reading
}
volatile int flag = 0;
volatile int count = 0;
```

```
void loop() {
  if (flag) {
    Serial.println("Count is " + String(count));
    flag = 0;
  }
}

void isr0() {
  flag = 1;
  count++;
  Serial.println("We have been interrupted.");
}
```

OUTPUT:

```
Count is 35
We have been interrupted.
Count is 36
We have been interrupted.
Count is 37
We have been interrupted.
Count is 38
We have been interrupted.
Count is 39
We have been interrupted.
Count is 40
We have been interrupted.
Count is 41
```

Experiment 3:

1. Install the library TimerOne-r11 into your Arduino library directory if you have not done so yet. See http://arduino.cc/en/Guide/Libraries

2. Try the example program ISRBlink. Read and understand it.

3. Write a program to generate a timer one interrupt every 100ms. Count them in an interrupt service routine. Every 1 second, print out the elapsed time on the serial console from your main loop.

4. Use the debounced pushbutton to generate an interrupt on pin 2. When you push the button once,

your system should keep counting time, but should stop printing it out. When you push the button again, the printing should restart. And so forth. (Demo #2).

CODE

```
#include <TimerOne.h>

//demo1
#define PIN_NUM 2
#define ANALOG_PIN A0

void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  pinMode(PIN_NUM, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_NUM), isr0, RISING);
  Timer1.initialize(100000);// 10Hz
  Timer1.attachInterrupt(timerIsr);

  pinMode(ANALOG_PIN, INPUT); // Configure pin A0 as an input for
analog reading
}
volatile int flag = 0;
volatile int count = 0;

volatile int timerCount = 0;
unsigned long secondCount = 0;

void loop() {
  if(timerCount >= 10) {
    secondCount+=10;
    timerCount=0;
     if (!flag) Serial.println(String(secondCount) + " interrupts
have passed.");
  }
}

void isr0() {
```

```
  flag++;
  if(flag >=2) flag=0;
  //Serial.println("We have been interrupted.");
}

void timerIsr() {
  timerCount++;
}
```

OUTPUT (button pressed after 40)

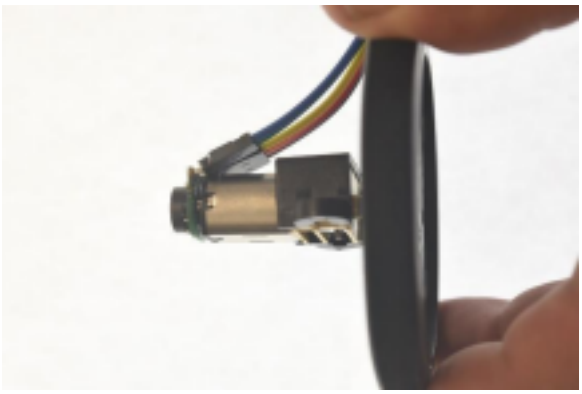
```
10 interrupts have passed.

20 interrupts have passed.

30 interrupts have passed.

40 interrupts have passed.

80 interrupts have passed.

90 interrupts have passed.

100 interrupts have passed.
```

Experiment 4:

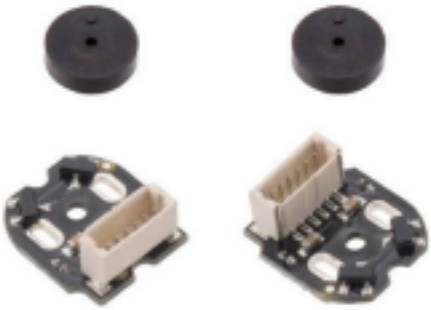
1. The wheel sensor and wheel need to be mounted to one of your motors. The sensors are very small parts and we have limited replacements. The soldering joints between the sensor board and the motor are very fragile. Be careful. I have already mounted these on each motor. Don't lose the magnetic disk on the rear shaft. You probably need to mount the wheel and tire. It is press on, but has a specific orientation.

2. The motor encoder board can be connected with a JST SH style connector to a breakout board. The breakout board can be inserted into a breadboard as shown below or connected with jumper wires. The pinout is identified in the drawing below. You can use the colors on the cable to identify signals or marking on the bottom of the breakout board.

3. The gearing is very high on these motors and thus trying to turn the wheel by hand will generally break the wheel mount. Please don't do this.



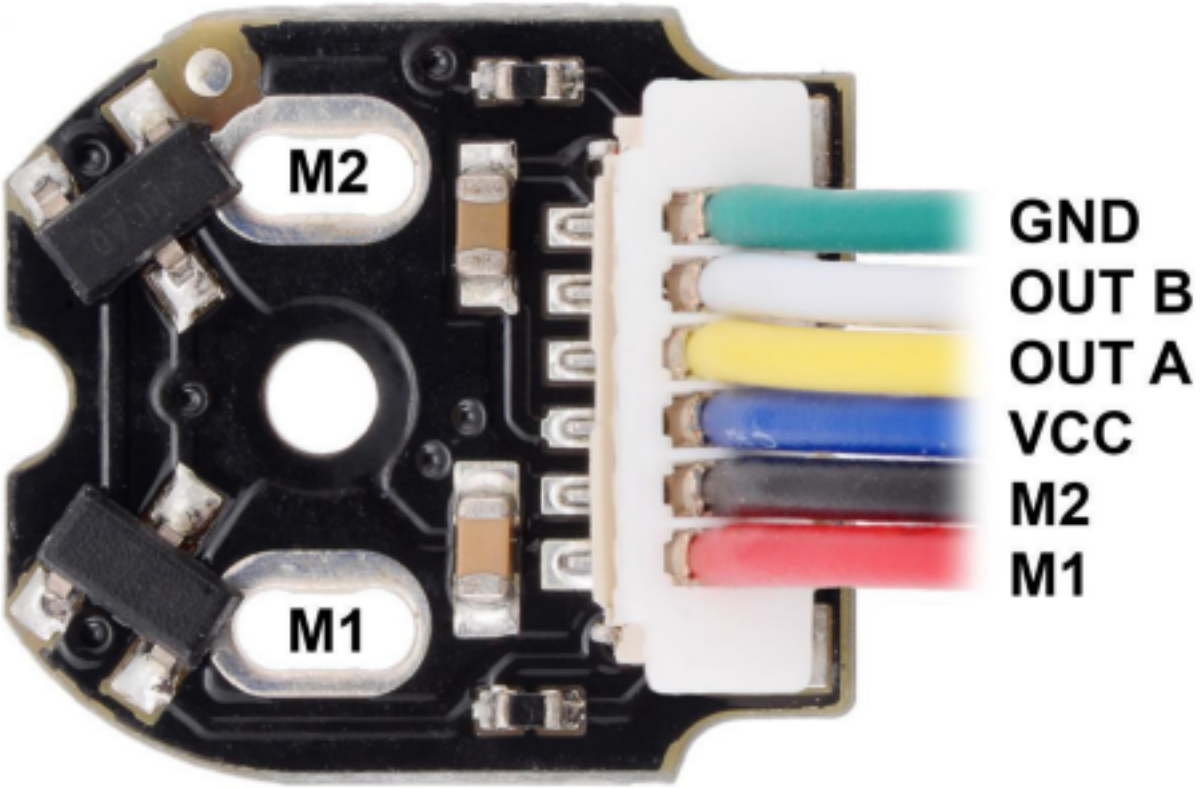
Assembled Motor/Wheel/Sensor (This picture uses a different connector)

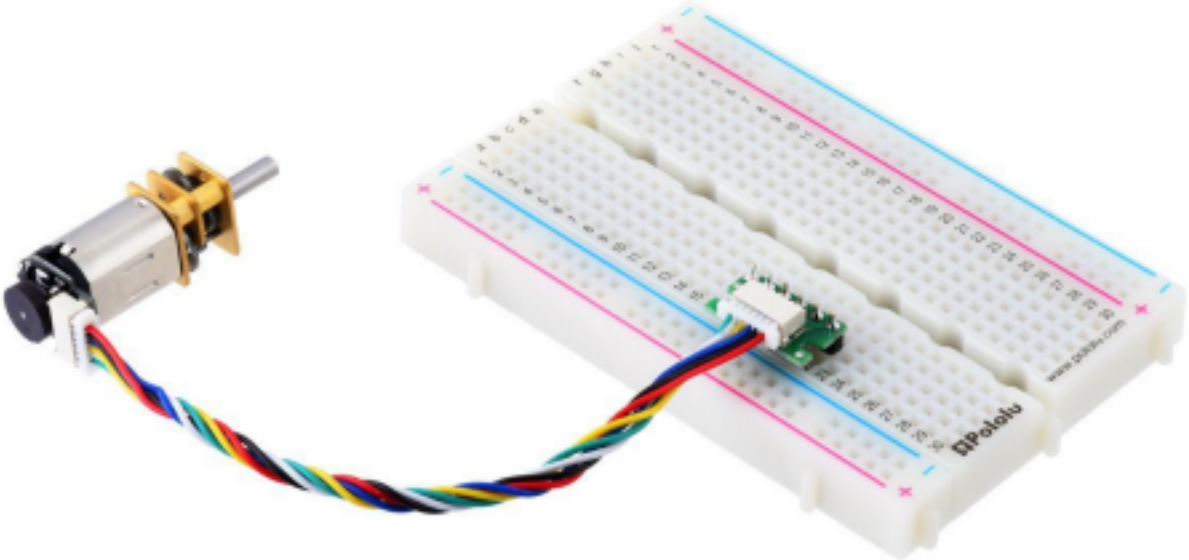


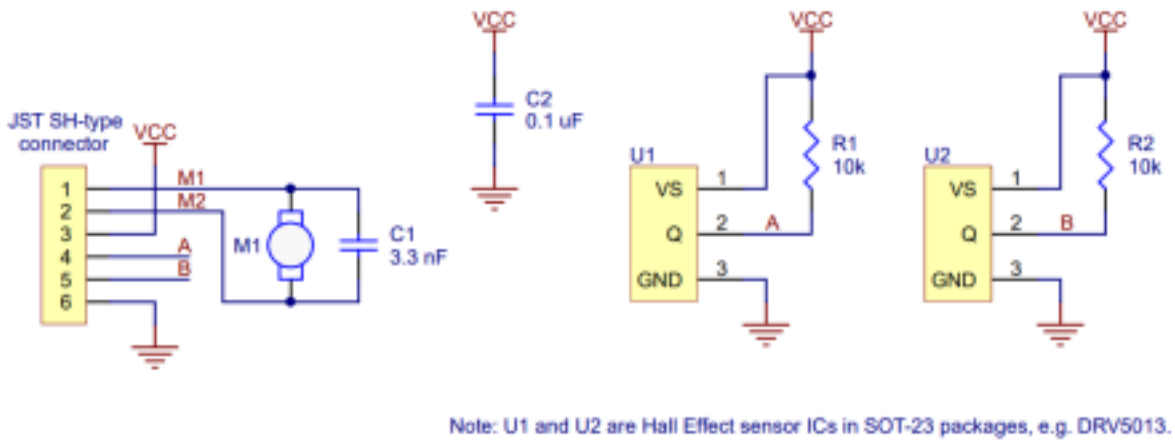
Shaft Encoders Wheel

Motor with encoder

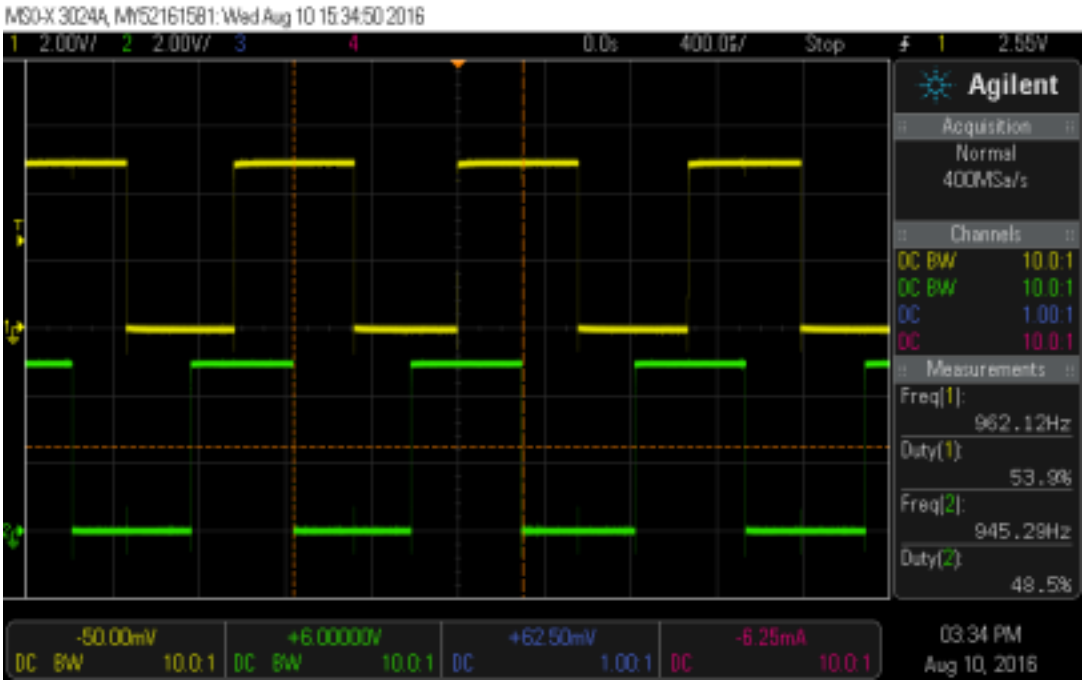
4. Using the breakout board and cable connect the VCC and ground on the encoder board to +5 and ground on the Arduino, respectively. Check carefully to make sure you have the correct pins. Connect channel A (Out A) to pin 2 of the Arduino and channel B (Out B) to pin 3.


M2
M1
GND
OUT B
OUT A
VCC
M2
M1





5. Write a program to generate an interrupt on every rising edge of channel A. The interrupt service routine should check the value on channel B. Based on the timing diagram below, increment a counter by one each time the wheel moves in one direction and generates a pulse on channel A and decrement the counter each time the wheel moves in the opposite direction and generates a pulse on channel A. Print out this counter value every time it changes.



CODE
`#include <TimerOne.h>`

```
#define PIN_CHANNEL_A 2
#define PIN_CHANNEL_B 3

void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  pinMode(PIN_CHANNEL_A, INPUT_PULLUP);
  pinMode(PIN_CHANNEL_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_CHANNEL_A), isr0, RISING);
  //Timer1.initialize(100000);// 10Hz
  //Timer1.attachInterrupt(timerIsr);
}
volatile int flag = 0;
volatile long count = 0;

//volatile int timerCount = 0;
long recentCount = 0;

void loop() {
  if(flag) {
    flag=0;
    int temp = count;
    if(temp != recentCount){
      recentCount=temp;
      Serial.println("Encoder count is " + String(recentCount));
    }
  }
}

void isr0() {
  int b = digitalRead(PIN_CHANNEL_B);
  if (b) count++;
  else count--;
  flag=1;
  //Serial.println("We have been interrupted.");
}
```

```
/*
void timerIsr()
{
  timerCount++;
}
*/
```

OUTPUT

```
Encoder count is 5981

Encoder count is 5989

Encoder count is 5997

Encoder count is 6005

Encoder count is 6013

Encoder count is 6021
```

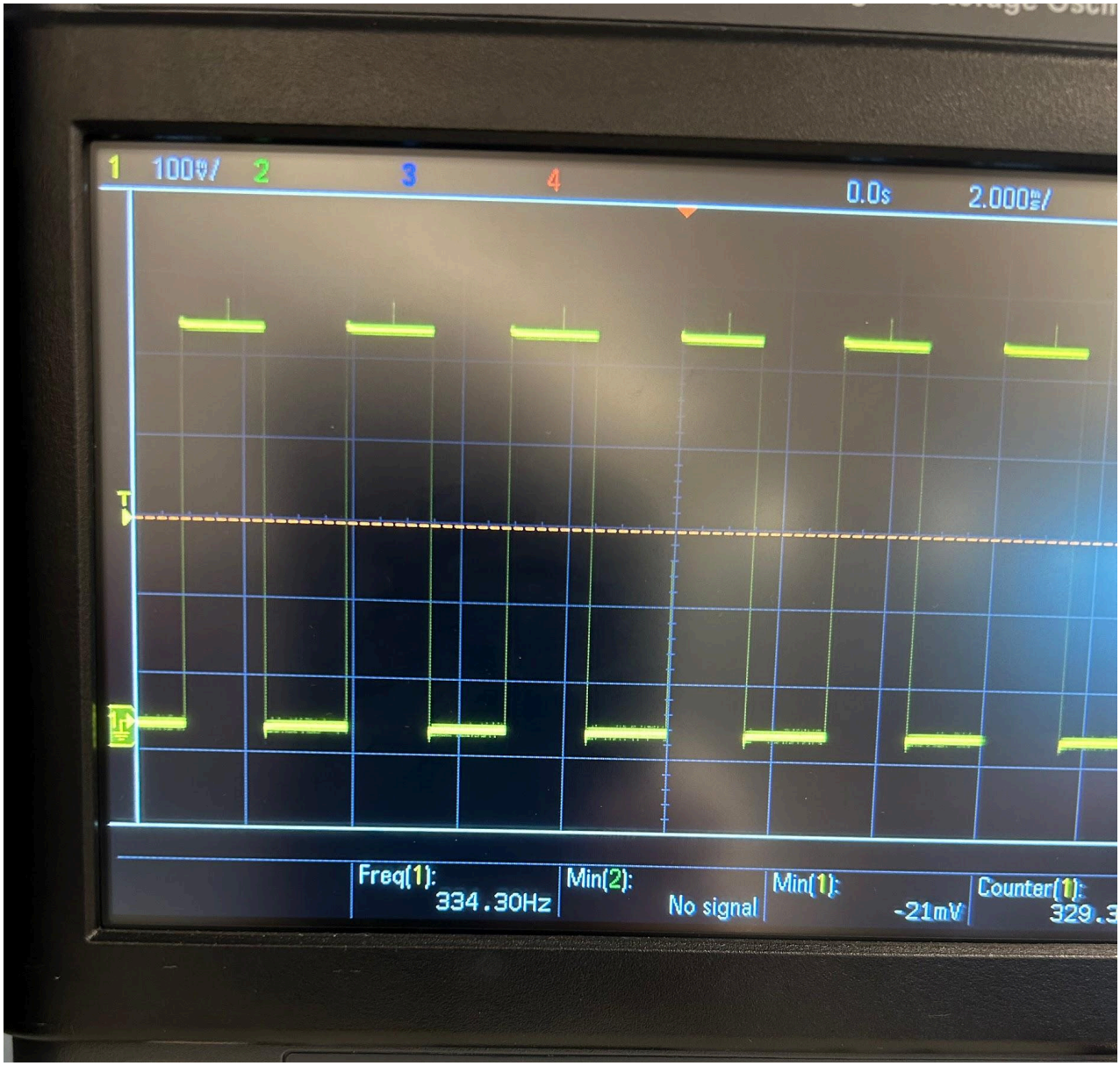
WHEN FLIPPED

```
Encoder count is -1267

Encoder count is -1275

Encoder count is -1284

Encoder count is -1292

Encoder count is -1300

Encoder count is -1308

Encoder count is -1316
```

6. Adjust the bench power supply to 2V. To operate the motor, connect pins M1 and M2 to the variable voltage output and ground (respectively) on a bench power supply. Adjust the voltage to have the wheel slowly turn. To reverse the motor, swap the M1 and M2 connections. You can run the M1 connection through another pushbutton switch on your protoboard if you want more control or you can use alligator clips.

7. Run this program. It should measure the distance travelled by the wheel in both directions. Determine how many counts correspond to one full wheel rotation. In your lab report, calculate the distance travelled per count. The wheel diameter is 60mm.

The distance travelled is ~.209mm per count. We started by measuring the frequency of the motor on the oscilloscope:



Running the motor for one full revolution (at 2 Volts) yielded approx 900 counts and took 2.69s

3:16
00:02.69
Reset
Start
Lap 1
00:02.69
World Clock
Alarms
Stopwatch
Timers

900/2.69 = 334.572 which was very close to our measured frequency.

With the wheel diameter being 60mm, the distance travelled of one revolution is 60*pi = ~188.5mm.

With our count of 900 being validated compared to the measured frequency, we then use that calculator distance/count:

188.5/900=0.209mm/count

Experiment 5:

1. Start with experiment 4. Using what you learned in experiment 3, add in code to make the LED on pin 13 blink for 200ms out of every 500ms based on a timer one interrupt.

2. Modify the pin 2 interrupt configuration so that the interrupt from channel A is detected on both the rising and falling edges. Make other modifications necessary so that the distance is tracked in both directions (positive in one direction, negative in the other).

3. Run this program. It should measure the distance travelled by the wheel in both directions. Determine how many counts correspond to one full wheel rotation. (Demo #3). In your lab report, calculate the distance travelled per count. The wheel diameter is 60mm.

Our counts per rev now were doubled since the interrupt triggered on both rising and falling edges. We also could no longer simply evaluate if (b) and had to check explicitly if (a==b) for counting since a and b would be equal on any edge change.

CODE

```
#include <TimerOne.h>
#include <math.h>

#define PIN_CHANNEL_A 2
#define PIN_CHANNEL_B 3
#define COUNTS_PER_REV 1800 //900 measured from previous, but now
we double count
                            // with both rising and falling edge
checking

void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  pinMode(PIN_CHANNEL_A, INPUT_PULLUP);
  pinMode(PIN_CHANNEL_B, INPUT_PULLUP);
  pinMode(LED_BUILTIN, OUTPUT);
  digitalWrite(LED_BUILTIN, HIGH);
  attachInterrupt(digitalPinToInterrupt(PIN_CHANNEL_A), isr0,
CHANGE); //any change
```

```
  Timer1.initialize(100000);// 10Hz
  Timer1.attachInterrupt(timerIsr);
}
volatile int flag = 0;
volatile long count = 0;

volatile int timerCount = 0;
long recentCount = 0;

void loop() {
  if(flag) {
    flag=0;
    int temp = count;
    if(temp != recentCount){
      recentCount=temp;
      //Serial.println("Encoder count is " + String(recentCount));
      float distance = float(recentCount) * (PI * 60.0f) / (float)
COUNTS_PER_REV;
      Serial.println("The distance is " + String(distance) + "mm");
    }
  }
}

void isr0() {
  int a = digitalRead(PIN_CHANNEL_A);
  int b = digitalRead(PIN_CHANNEL_B);
  if (a==b) count++;
  else count--;
  flag=1;
  //Serial.println("We have been interrupted.");
}


void timerIsr()
{
  timerCount++;
  if(timerCount < 2) digitalWrite(LED_BUILTIN, HIGH);
  else digitalWrite(LED_BUILTIN, LOW);
```

```
  if(timerCount >=5) timerCount=0;
}
```

OUTPUT (for 5 revolutions, we expect 188.5 * 5 = 942.5)

(5 revolutions)

```
The distance is -920.80mm

The distance is -922.79mm

The distance is -924.68mm

The distance is -926.67mm

The distance is -928.55mm

The distance is -930.54mm

The distance is -932.53mm

The distance is -934.41mm

The distance is -936.40mm

The distance is -938.29mm

The distance is -940.28mm

The distance is -942.16mm
```

(Just showing it goes the other way)

```
The distance is -879.33mm

The distance is -877.45mm

The distance is -875.46mm

The distance is -873.57mm

The distance is -871.58mm

The distance is -869.70mm
```